



ALARM CLOCK

Introduction:

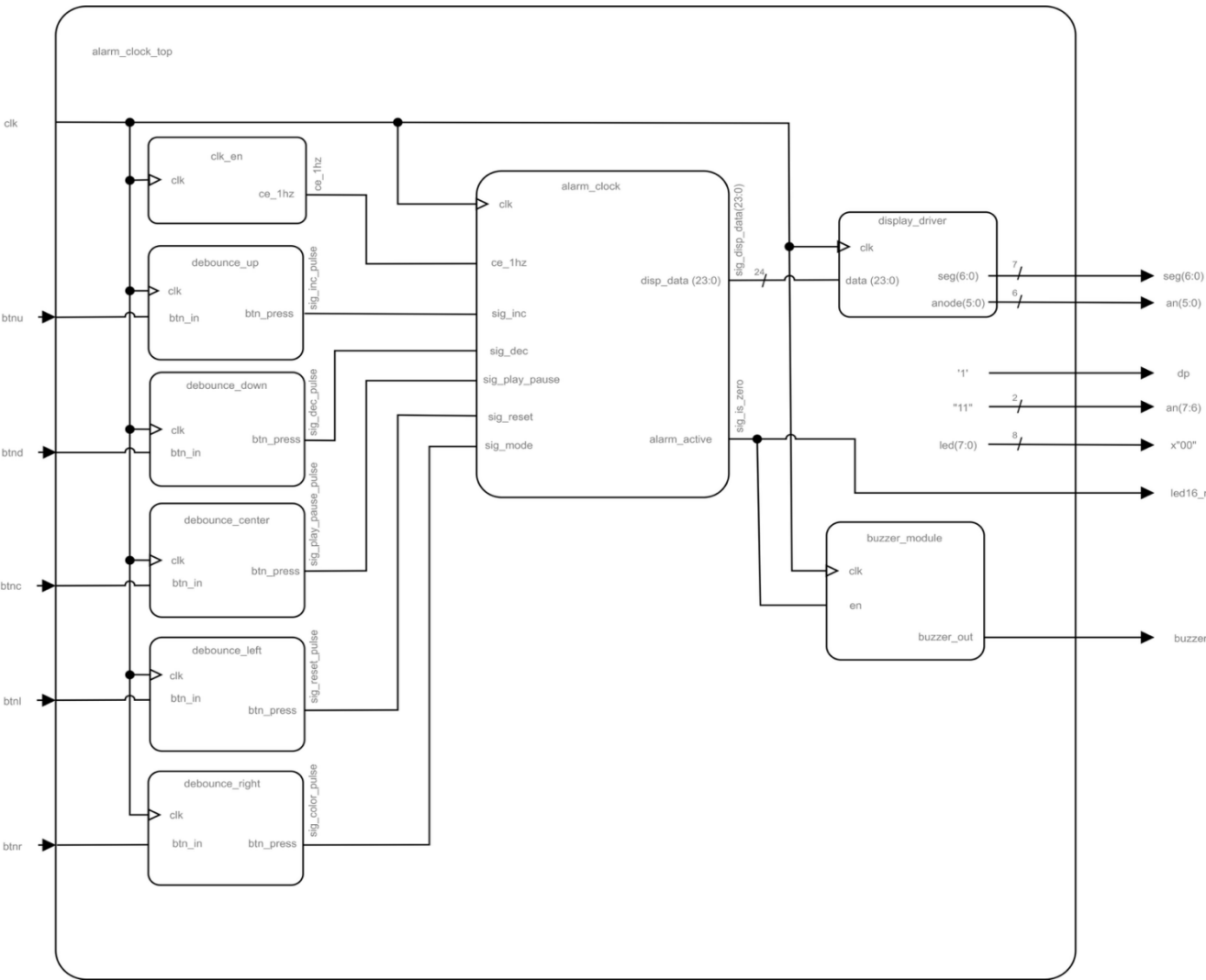
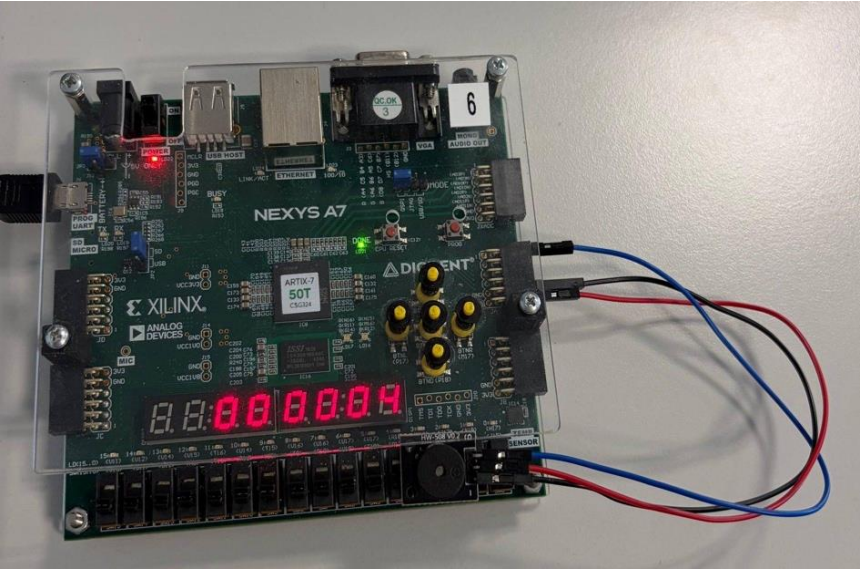
This project presents the implementation of a fully functional, adjustable alarm clock on the **Nexys A7-50T FPGA** board using the VHDL hardware description language. The project demonstrates digital circuit design, modular architecture, and FPGA peripheral integration.

Main Objective:

Creation of a working alarm clock with a maximum setting of 24 hours (23.59. 59). When the timer reaches zero, a visual (RGB LED) and audible (external buzzer) alarm triggers. The user can adjust time increments (seconds, minutes, hours) and stop/reset the alarm as needed.

Interactive Components:

- ❖ **6x 7-Segment Displays:** Used to show time in HH.MM.SS format.
- ❖ **User Interface (5 Buttons):**
 - **BTNU (Up):** Increments the selected time value.
 - **BTND (Down):** Decrements the selected time value.
 - **BTNC (Center):** Starts/Stops the countdown; silences the alarm.
 - **BTNL (Left):** Resets the time to 00.00.00.
 - **BTNR (Right):** Cycles through adjustment modes (Sec -> Min -> Hr).
- ❖ **RGB LED:** Lights up when time run out.
- ❖ **Buzzer (HW508):** Connected to the **JA Pmod** port (Pin C17). It generates a 50% duty cycle square wave during the alarm.



Module Hierarchy (alarm_clock_top):

The design follows a modular approach, combining custom logic with standard components:

alarm_clock: Manages time storage, setting logic, countdown execution, and the *alarm_active* signal generation.

debounce instantiations: Each button has individual *debounce* instance to ensure stable operation and prevent double-triggering.

clock_en: Derives a precise 1 Hz pulse from the main board oscillator.

display_driver: Displays time in 6x 7-segments.

buzzer_module: Generates square waveform signal frequency for the buzzer when *alarm_active* is high.

Team members:

- | | |
|--------------|-----------------------|
| Patrik Malý | - Code, documentation |
| Radek Ondra | - Top level scheme |
| Assiya Murat | - Poster |

Github

